

MODULE 2 LAB WORKBOOK

Unit Test Frameworks

Name – Sristi Saha

Institute – Institute of Engineering & Management, Kolkata

Enrollment No. – 12023052016066

Module:	Unit Test Frameworks
Module Focus:	Unit testing, PyTest, fixtures, conftest.py, HTML reporting, data-driven testing and Page Object Model
Language:	Python
Browser:	Google Chrome
Automation Framework:	Selenium WebDriver
Experiments Covered:	Assignment 7 to Assignment 9

Experiment List

Experiment	Experiment Title	Primary Concepts
7	Page Object Model (POM) Restructure	Page classes, locators, UI methods, separate assertions
8	Data-Driven Automation (DDT)	External CSV data, multiple login cases, validation assertions
9	PyTest Integration with HTML Reporting	Fixtures, conftest.py, PyTest, HTML report, failure screenshots

Experiment 7: Page Object Model (POM) Restructure

Module Unit Test Frameworks

Experiment Title Page Object Model (POM) Restructure

Practice Website <https://www.saucedemo.com/>

Problem Statement

Take a working Selenium login script and restructure it using the Page Object Model design pattern. The page classes must contain locator definitions and UI interaction methods, while the actual test assertions remain separate from the locators and page implementation.

Objective

To understand the Page Object Model pattern and create a maintainable Selenium structure by separating page-specific UI operations from test validation logic.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm
- Page Object Model (POM)
- Page classes and locators
- WebDriverWait and expected_conditions

Implementation Steps

1. Create a LoginPage class containing the username, password, login button and error-message locators.
2. Create methods in LoginPage for entering the username, entering the password, clicking Login and reading the error message.
3. Create a DashboardPage class containing the Products-page locator and UI methods.
4. Create the test script separately and initialize the page objects.
5. Perform a valid login through the page methods.
6. Assert that the Products page is displayed and that the page title is Products.
7. Close the browser after the test is completed.

Source Code —

login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
class LoginPage:
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")
```

```
ERROR_MESSAGE = (By.CSS_SELECTOR, "[data-test='error']")
```

```
def __init__(self, driver):  
    self.driver = driver  
    self.wait = WebDriverWait(driver, 10)
```

```
def enter_username(self, username):  
    element = self.wait.until(  
        EC.visibility_of_element_located(self.USERNAME)  
    )  
    element.clear()  
    element.send_keys(username)
```

```
def enter_password(self, password):  
    element = self.wait.until(  
        EC.visibility_of_element_located(self.PASSWORD)  
    )  
    element.clear()  
    element.send_keys(password)
```

```
def click_login(self):  
    self.wait.until(  
        EC.element_to_be_clickable(self.LOGIN_BUTTON)  
    ).click()
```

```
def login(self, username, password):  
    self.enter_username(username)  
    self.enter_password(password)  
    self.click_login()
```

```
def get_error_message(self):  
    return self.wait.until(  
        EC.visibility_of_element_located(self.ERROR_MESSAGE)  
    ).text
```

dashboard_page.py

```
from selenium.webdriver.common.by import By  
from selenium.webdriver.support.ui import WebDriverWait  
from selenium.webdriver.support import expected_conditions as EC
```

```
class DashboardPage:  
    PRODUCTS_TITLE = (By.CLASS_NAME, "title")  
  
    def __init__(self, driver):  
        self.driver = driver  
        self.wait = WebDriverWait(driver, 10)  
  
    def get_page_title(self):  
        return self.wait.until(  
            EC.visibility_of_element_located(self.PRODUCTS_TITLE)  
        ).text
```

```
def is_dashboard_displayed(self):
    try:
        return self.wait.until(
            EC.visibility_of_element_located(self.PRODUCTS_TITLE)
        ).is_displayed()
    except:
        return False
```

test_login.py

```
from selenium import webdriver
from login_page import LoginPage
from dashboard_page import DashboardPage
import time

driver = webdriver.Chrome()
try:
    driver.get("https://www.saucedemo.com/")
    driver.maximize_window()
    login_page = LoginPage(driver)
    dashboard_page = DashboardPage(driver)
    login_page.login("standard_user", "secret_sauce")

    assert dashboard_page.is_dashboard_displayed(), \
        "Dashboard was not displayed after valid login."

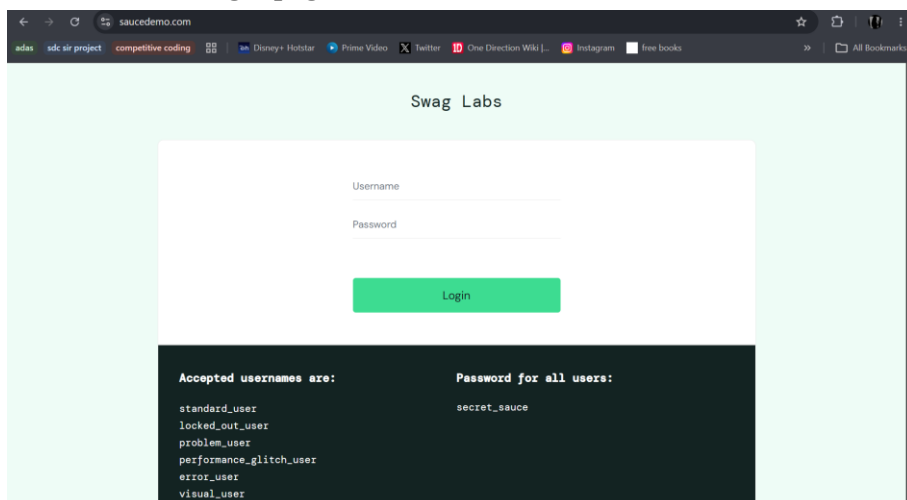
    assert dashboard_page.get_page_title() == "Products", \
        "Products title was not displayed."

    print("TEST 1 PASSED: Valid login successful.")

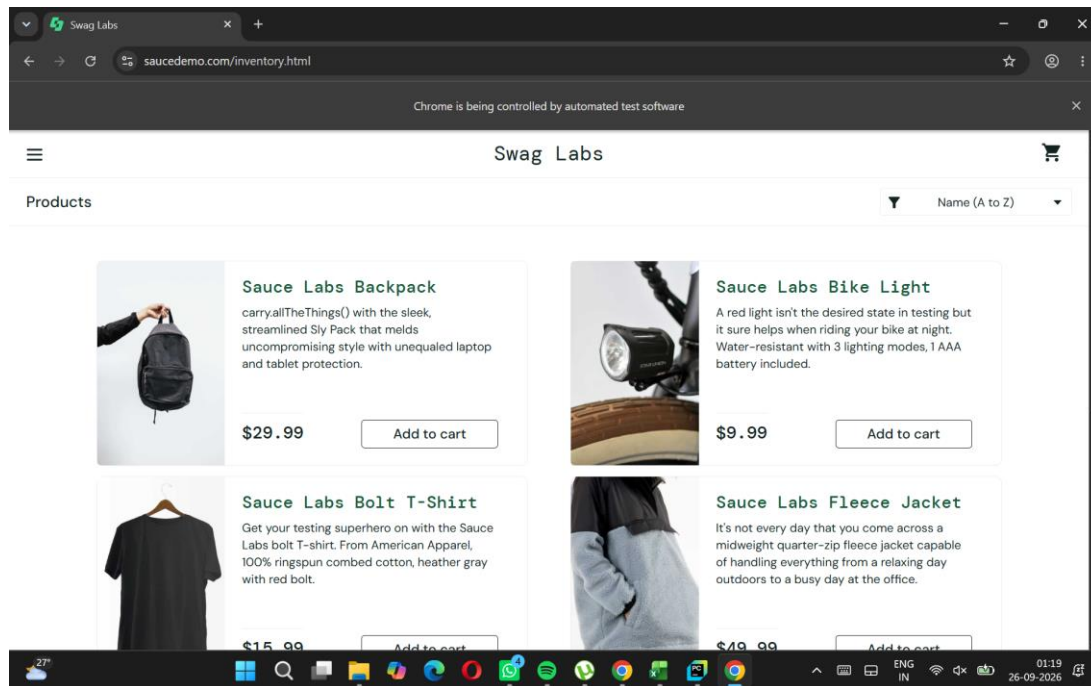
finally:
    time.sleep(5)
    driver.quit()
```

Output

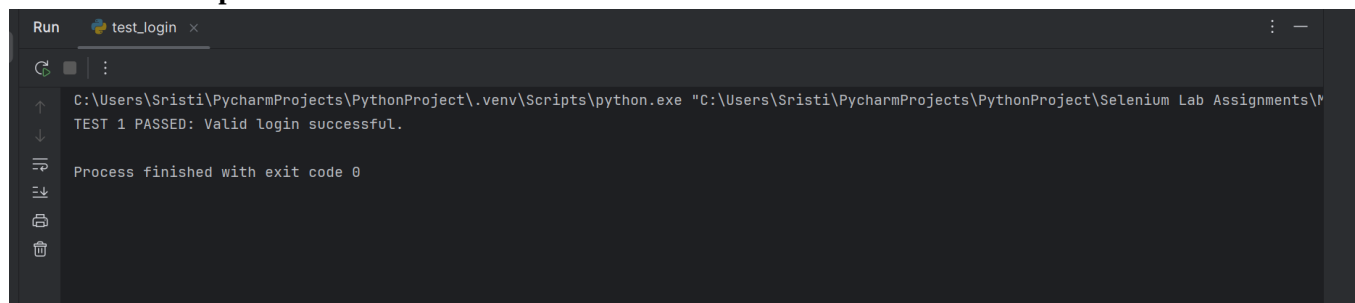
7.1 SauceDemo login page before execution



7.2 Successful login - Products page



7.3 Terminal output



Result and Observation

The Selenium login automation was successfully restructured into separate page classes. The LoginPage and DashboardPage classes contain locators and UI methods, while the test script contains the assertions. The test successfully navigated to the Products page.

Conclusion

The experiment demonstrated the Page Object Model design pattern and showed how separating page implementation from test assertions improves the structure and maintainability of Selenium automation.

Experiment 8: Data-Driven Automation (DDT)

Module Unit Test Frameworks / Data-Driven Automation

Experiment Title Data-Driven Login Automation using External CSV Test Data

Practice Website <https://www.saucedemo.com/>

Problem Statement

Build a Selenium login script that reads multiple test cases from an external source. The automation must execute valid and invalid username-password combinations and verify that the expected success or validation error is produced for each test case.

Objective

To implement data-driven testing by separating test data from the automation logic and executing multiple login scenarios from an external CSV file.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm
- CSV file and csv.DictReader
- Page Object Model
- Data-Driven Testing (DDT)

Implementation Steps

1. Create an external test_data.csv file containing the test case name, username, password and expected result.
2. Read the CSV file using csv.DictReader.
3. Loop through every test-data row.
4. Open the SauceDemo login page for each test case.
5. Submit the username and password using LoginPage.
6. For a successful case, verify that the browser reaches the inventory page.
7. For an error case, read the displayed error message and compare it with the expected message.
8. Print PASS or FAIL for every data row and close the browser.

Source Code —

test_data.csv

```
test_case,username,password,expected_result,expected_error
```

```
Valid Login,standard_user,secret_sauce,success,
```

```
Invalid Username,wrong_user,secret_sauce,error,Epic sadface: Username and password do not match any user in this service
```

```
Invalid Password,standard_user,wrong_password,error,Epic sadface: Username and password do not match any user in this service
```

```
Both Invalid,wrong_user,wrong_password,error,Epic sadface: Username and password do not match any user in this service
```

```
Locked User,locked_out_user,secret_sauce,error,"Epic sadface: Sorry, this user has been locked out."
```

login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")
    ERROR_MESSAGE = (By.CSS_SELECTOR, "[data-test='error']")

    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 15)

    def enter_username(self, username):
        element = self.wait.until(
            EC.visibility_of_element_located(self.USERNAME)
        )
        element.clear()
        element.send_keys(username)

    def enter_password(self, password):
        element = self.wait.until(
            EC.visibility_of_element_located(self.PASSWORD)
        )
        element.clear()
        element.send_keys(password)

    def click_login(self):
        button = self.wait.until(
            EC.element_to_be_clickable(self.LOGIN_BUTTON)
        )
        button.click()

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

    def get_error_message(self):
        return self.wait.until(
            EC.visibility_of_element_located(self.ERROR_MESSAGE)
        ).text
```

data_driven_login.py

```
import csv
from selenium import webdriver
from selenium.webdriver.support.ui import WebDriverWait
from login_page import LoginPage
```

with open("test_data.csv", newline="", encoding="utf-8-sig") as file:

reader = csv.DictReader(file)

for row in reader:

test_case = row["test_case"]

username = row["username"]

password = row["password"]

expected_result = row["expected_result"]

expected_error = row["expected_error"]

print("\n-----")

print("Running:", test_case)

print("Username:", username)

print("Expected Result:", expected_result)

driver = webdriver.Chrome()

try:

driver.maximize_window()

driver.get("https://www.saucedemo.com/")

Wait until page is completely loaded

WebDriverWait(driver, 15).until(

lambda d: d.execute_script(

"return document.readyState"

) == "complete"

)

print("Page Title:", driver.title)

print("Current URL:", driver.current_url)

login_page = LoginPage(driver)

login_page.login(username, password)

if expected_result == "success":

assert "inventory" in driver.current_url, \

"User was not redirected to the inventory page."

print("RESULT: PASS")

elif expected_result == "error":

actual_error = login_page.get_error_message()

assert expected_error in actual_error, \

f"Expected: {expected_error}\nActual: {actual_error}"

print("RESULT: PASS")

print("Actual Error:", actual_error)

except AssertionError as e:

print("RESULT: FAIL")

print("Assertion Error:", e)

except Exception as e:

print("RESULT: FAIL")

print("Unexpected Error:", type(e).__name__)

print("Error Details:", e)

finally:

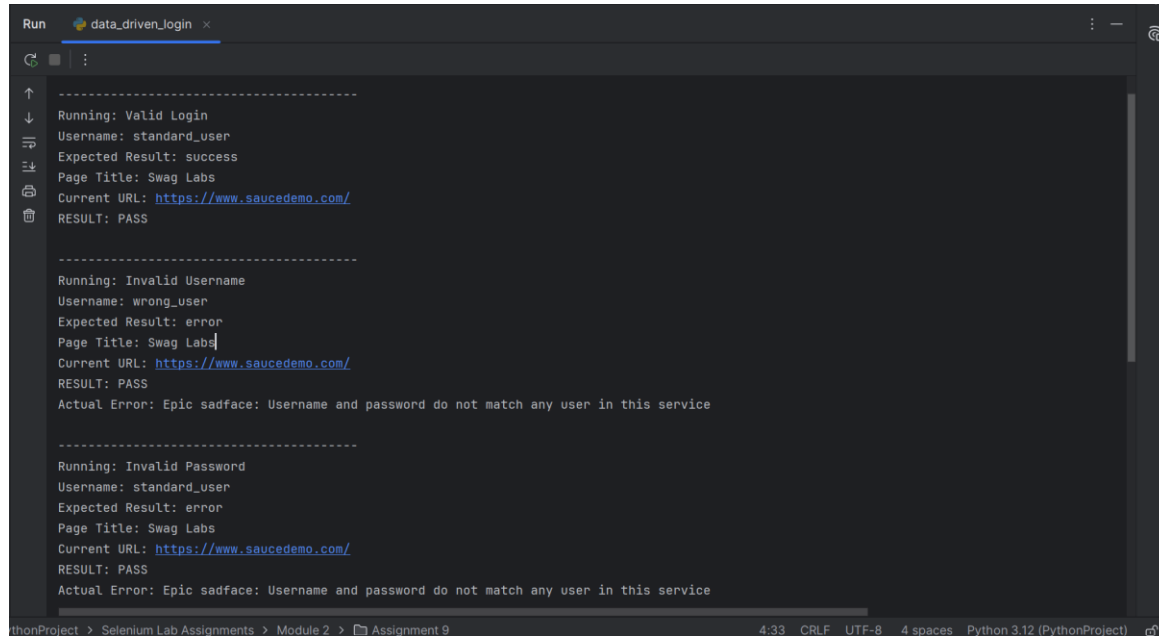
driver.quit()

print("\n=====")

```
print("DATA-DRIVEN TESTING COMPLETED")
print("=====")
```

Output

8.1 Console showing all data-driven test cases and PASS results



```
Run data_driven_login x
-----
Running: Valid Login
Username: standard_user
Expected Result: success
Page Title: Swag Labs
Current URL: https://www.saucedemo.com/
RESULT: PASS

-----
Running: Invalid Username
Username: wrong_user
Expected Result: error
Page Title: Swag Labs
Current URL: https://www.saucedemo.com/
RESULT: PASS
Actual Error: Epic sadface: Username and password do not match any user in this service

-----
Running: Invalid Password
Username: standard_user
Expected Result: error
Page Title: Swag Labs
Current URL: https://www.saucedemo.com/
RESULT: PASS
Actual Error: Epic sadface: Username and password do not match any user in this service

-----
Running: Both Invalid
Username: wrong_user
Expected Result: error
Page Title: Swag Labs
Current URL: https://www.saucedemo.com/
RESULT: PASS
Actual Error: Epic sadface: Username and password do not match any user in this service

-----
Running: Locked User
Username: locked_out_user
Expected Result: error
Page Title: Swag Labs
Current URL: https://www.saucedemo.com/
RESULT: PASS
Actual Error: Epic sadface: Sorry, this user has been locked out.

=====
DATA-DRIVEN TESTING COMPLETED
=====

Process finished with exit code 0
```

Result and Observation

The script successfully read the login scenarios from the external CSV file and executed them one by one. The valid login case reached the inventory page, while invalid username, invalid password, both-invalid and locked-user cases displayed the expected validation errors. All five data-driven cases passed.

Conclusion

The experiment demonstrated how external test data can be separated from Selenium automation logic. Data-driven testing makes it possible to add or modify login scenarios without rewriting the main test flow.

Experiment 9: PyTest Integration with HTML Reporting

Module PyTest Framework

Experiment Title PyTest Integration with HTML Reporting and Failure Screenshots

Practice Website <https://www.saucedemo.com/>

Problem Statement

Convert the Selenium setup to run using PyTest. Browser initialization and teardown must be handled using a PyTest fixture. The test suite must be executable from the terminal and generate an HTML report. A screenshot must be captured automatically when a test fails.

Objective

To use PyTest fixtures for Selenium browser management, execute multiple tests from the terminal, generate a self-contained HTML report and preserve screenshot evidence for failed test cases.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm
- PyTest
- pytest-html
- Assertions
- HTML test reporting

Implementation Steps

1. Install pytest and pytest-html.
2. Create a conftest.py file containing the driver fixture.
3. Open the SauceDemo website in the fixture and close the browser after each test.
4. Create test methods using the LoginPage page object.
5. Add valid-login and invalid-login tests.
6. Add one intentionally failing assertion so that both PASS and FAIL results appear in one execution.
7. Use pytest_runtest_makereport to capture a screenshot when a test fails.
8. Run the test suite from the terminal with pytest --html=report.html --self-contained-html.
9. Open report.html and verify the test results and failure evidence.

Source Code —

pages/login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
class LoginPage:
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
```

```

LOGIN_BUTTON = (By.ID, "login-button")
ERROR_MESSAGE = (By.CSS_SELECTOR, "[data-test='error']")

def __init__(self, driver):
    self.driver = driver
    self.wait = WebDriverWait(driver, 10)

def enter_username(self, username):
    element = self.wait.until(
        EC.visibility_of_element_located(self.USERNAME)
    )
    element.clear()
    element.send_keys(username)

def enter_password(self, password):
    element = self.wait.until(
        EC.visibility_of_element_located(self.PASSWORD)
    )
    element.clear()
    element.send_keys(password)

def click_login(self):
    self.wait.until(
        EC.element_to_be_clickable(self.LOGIN_BUTTON)
    ).click()

def login(self, username, password):
    self.enter_username(username)
    self.enter_password(password)
    self.click_login()

def get_error_message(self):
    return self.wait.until(
        EC.visibility_of_element_located(self.ERROR_MESSAGE)
    ).text

```

conftest.py

```

import pytest
from pathlib import Path
from selenium import webdriver

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://www.saucedemo.com/")
    yield driver
    driver.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield

```

```

report = outcome.get_result()

if report.when == "call" and report.failed:
    driver = item.funcargs.get("driver")

    if driver:
        screenshot_dir = Path("screenshots")
        screenshot_dir.mkdir(exist_ok=True)
        screenshot_path = screenshot_dir / f"{item.name}.png"
        driver.save_screenshot(str(screenshot_path))
        print(f"\nScreenshot saved: {screenshot_path}")
        pytest_html = item.config.pluginmanager.getplugin("html")

        if pytest_html:
            extra = getattr(report, "extras", [])
            extra.append(pytest_html.extras.image(str(screenshot_path)))
            report.extras = extra

```

tests/test_login.py

```

from pages.login_page import LoginPage

```

```

def test_valid_login(driver):
    login_page = LoginPage(driver)
    login_page.login(
        "standard_user",
        "secret_sauce"
    )
    assert "inventory" in driver.current_url

def test_invalid_username(driver):
    login_page = LoginPage(driver)
    login_page.login(
        "wrong_user",
        "secret_sauce"
    )
    error = login_page.get_error_message()
    assert "Username and password do not match" in error

def test_invalid_password(driver):
    login_page = LoginPage(driver)
    login_page.login(
        "standard_user",
        "wrong_password"
    )
    error = login_page.get_error_message()
    assert "Username and password do not match" in error

def test_intentional_failure_for_screenshot(driver):
    """
    Intentionally fails so one execution contains both PASS and FAIL.
    The failure triggers the screenshot hook in conftest.py.
    """

```

```

login_page = LoginPage(driver)
login_page.login(
    "standard_user",
    "secret_sauce"
)
assert "THIS_PAGE_DOES_NOT_EXIST" in driver.current_url

```

pytest.ini

```

[pytest]
testpaths = tests
addopts = -v

```

Execution Command

```
pytest --html=report.html --self-contained-html
```

Output

9.1 Terminal output showing 3 PASSED and 1 FAILED

```

Terminal Local x + v

(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 2> pytest --html=report.html --self-contained-html
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 2
plugins: anyio-4.10.0, html-4.2.0, metadata-3.1.1
collecting ...
DevTools listening on ws://127.0.0.1:58796/devtools/browser/9d53ab5d-8cc0-4ee4-bac8-181fe5c3652e
[11220:9080:0926/021250.300:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
collected 4 items

Assignment9_PyTest\tests\test_logins.py
DevTools listening on ws://127.0.0.1:52990/devtools/browser/401e1f66-7c3c-48cf-8a03-e6bb128c3923
.
DevTools listening on ws://127.0.0.1:49261/devtools/browser/db59c5ef-57b5-4489-b658-4e25d03011ab
.
DevTools listening on ws://127.0.0.1:51214/devtools/browser/3054cb8e-1ac4-4b92-a597-2df78efd9ed9
.
DevTools listening on ws://127.0.0.1:62464/devtools/browser/8d72092a-c30f-4f25-ab3b-867d08562d70
.
Screenshot saved: screenshots\test_intentional_failure_for_screenshot.png
F [100%]

===== FAILURES =====
----- test_intentional_failure_for_screenshot -----

driver = <selenium.webdriver.chrome.webdriver.WebDriver (session="3a2dec05f9059fe714523e5ff9fc964f")>

def test_intentional_failure_for_screenshot(driver):
    """
    Intentionally fails so one execution contains both PASS and FAIL.
    The failure triggers the screenshot hook in conftest.py.
    """
    login_page = LoginPage(driver)

def test_intentional_failure_for_screenshot(driver):
    """
    Intentionally fails so one execution contains both PASS and FAIL.
    The failure triggers the screenshot hook in conftest.py.
    """
    login_page = LoginPage(driver)

    login_page.login(
        "standard_user",
        "secret_sauce"
    )

    # Deliberately incorrect assertion
    > assert "THIS_PAGE_DOES_NOT_EXIST" in driver.current_url
E   assert 'THIS_PAGE_DOES_NOT_EXIST' in 'https://www.saucedemo.com/inventory.html'
E   + where 'https://www.saucedemo.com/inventory.html' = <selenium.webdriver.chrome.webdriver.WebDriver (session="3a2dec05f9059fe714523e5ff9fc964f")>.current_url

Assignment9_PyTest\tests\test_logins.py:54: AssertionError

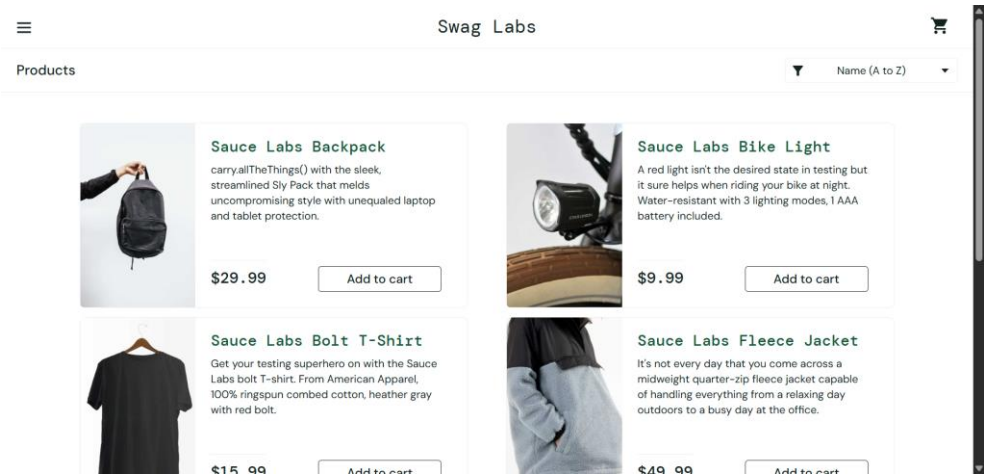
===== warnings summary =====
Assignment9_PyTest\tests\test_logins.py::test_intentional_failure_for_screenshot
  C:\Users\Sristi\PycharmProjects\PythonProject\.venv\Lib\site-packages\pytest_html\selfcontained_report.py:31: UserWarning: Self-contained HTML report includes link to external resource: screenshots\test_intentional_failure_for_screenshot.png
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
----- Generated html report: file:///C:/Users/Sristi/PycharmProjects/PythonProject/Selenium%20Lab%20Assignments/Module%202/report.html -----
===== short test summary info =====
FAILED Assignment9_PyTest\tests\test_logins.py::test_intentional_failure_for_screenshot - assert 'THIS_PAGE_DOES_NOT_EXIST' in 'https://www.saucedemo.com/inventory.html'

===== 1 failed, 3 passed, 1 warning in 37.22s =====

```

9.2 Browser screenshot automatically captured for the failed test



9.3 HTML report showing the test results

report.html

Report generated on 26-Sep-2026 at 02:13:20 by pytest-html v4.2.0

Environment

Python	3.12.4
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• anyio: 4.10.0 • html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-25

Summary

4 tests took 00:00:37.

(Un)check the boxes to filter the results.

☒ 1 Failed, ☒ 3 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns, ☐ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Failed	Assignment9_PyTest/tests/test_logins.py::test_intentional_failure_for_screenshot	00:00:06	
<pre>driver = <selenium.webdriver.chrome.webdriver.WebDriver (session="3a2dec05f9059fe714523e5ff9fc964f")> def test_intentional_failure_for_screenshot(driver): """ Intentionally fails so one execution contains both PASS and FAIL. The failure triggers the screenshot hook in conftest.py. login_page = LoginPage(driver) login_page.login("standard_user", "secret_sauce") # Deliberately incorrect assertion. assert "THIS_PAGE_DOES_NOT_EXIST" in driver.current_url ></pre>			
Passed	Assignment9_PyTest/tests/test_logins.py::test_valid_login	00:00:06	
Passed	Assignment9_PyTest/tests/test_logins.py::test_invalid_username	00:00:08	
Passed	Assignment9_PyTest/tests/test_logins.py::test_invalid_password	00:00:06	

Result and Observation

The Selenium tests were executed through PyTest using a reusable driver fixture. Three login tests passed and the intentionally incorrect assertion failed. The failure hook created a screenshots folder and saved a PNG image of the browser state at the time of failure. The pytest-html plugin generated an HTML report containing the execution results.

Conclusion

The experiment demonstrated the use of PyTest fixtures, conftest.py, assertions and HTML reporting with Selenium. Automatic screenshot capture on failure provides useful visual evidence for debugging and test documentation.

Overall Module Result

The Unit Test Frameworks, pytest Framework and Page Object Model work was completed through three practical experiments. The experiments progressed from restructuring Selenium automation with POM, to executing multiple external data-driven login cases, and finally to running Selenium tests through pytest with fixtures, HTML reporting and automatic failure screenshots.

Overall Observation

- Page Object Model separates page locators and UI operations from test assertions.
- External CSV data allows multiple test cases to be executed without changing the automation logic.
- pytest fixtures provide reusable browser initialization and teardown.
- conftest.py can contain common test configuration and failure-handling logic.
- pytest-html provides an HTML representation of test execution results.
- Failure screenshots provide visual evidence of the browser state when an assertion fails.

Overall Conclusion

The experiments provided practical understanding of structured Selenium automation using Python. The completed work demonstrates the use of Page Object Model, data-driven testing and pytest-based execution with reporting. These concepts provide a foundation for developing maintainable, reusable and testable browser automation frameworks.